Cosmos 组件嵌入 Qt 宿主窗口接入说明

> **适用对象**：需要在 Qt 应用中嵌入 Cosmos 组件窗口的开发同事
> **示例代码位置**：`CosmosQtHost/MainWindow.cpp`

---

1. 整体原理概述

一句话概括：
Cosmos 组件在自己的进程 / GUI 框架中创建一个原生窗口（Windows 上是 HWND），通过 SDK 把这个窗口的句柄（WindowHandle）返回给 Qt 宿主；Qt 这边再用 `QWindow::fromWinId` + `QWidget::createWindowContainer` 把这个外部原生窗口托管到我们的 Qt 界面中，看起来就像普通的 Qt 控件一样。

关键点：

- **父窗口**：Qt 宿主提供一个 `QWidget` 作为“容器”，通过 `winId()` 拿到底层原生窗口句柄，作为 Cosmos 创建子窗口的父窗口。
- **子窗口**：Cosmos 组件内部创建自己的窗口，并把窗口句柄以字符串形式通过 SDK 结果返回。
- **嵌入桥梁**：Qt 使用 `QWindow::fromWinId(WindowHandle)` 接管这个已有的原生窗口，再用 `QWidget::createWindowContainer` 包装成 `QWidget`，加入 Qt 布局中。

---

2. 关键对象与数据流

- **Qt 侧**
  - `QWidget* m_cosmosContainer`：嵌入区域的容器控件。
  - `WId parentWindowId`：`m_cosmosContainer->winId()`，Qt 容器对应的原生窗口 ID。
  - `QWindow* m_embeddedWindow`：对外部 Cosmos 窗口的 Qt 封装。
  - `QWidget* m_embeddedWidget`：`createWindowContainer` 生成的包装控件，真正放进布局。
- **Cosmos 侧（通过 SDK）**
  - `CreateWidget`：创建组件窗口的 RPC 方法。
  - `ParentHandle`：Qt 提供的父窗口句柄字符串。
  - `WindowHandle`：Cosmos 返回的子窗口句柄字符串（要嵌入的那个窗口）。
  - `WidgetHandle`：逻辑层面的组件句柄，用于后续销毁等操作。

数据流简述：

1. Qt 先拿到自己的容器窗口句柄 `ParentHandle`。
2. Qt 通过 `Cosmos_Invoke("CreateWidget", 参数 JSON)` 调用 Cosmos，引擎拿到 `ParentHandle` 后创建子窗口并设为其子窗口。
3. Cosmos 把 `WindowHandle`（子窗口原生句柄）返回给 Qt。
4. Qt 用 `QWindow::fromWinId(WindowHandle)` 包装这个外部窗口，再用 `QWidget::createWindowContainer` 放入 Qt 布局，实现“嵌入”。

---

3. 嵌入流程分步说明

3.1 准备 Qt 宿主容器

在主窗口构造函数中准备一个专门用于嵌入的容器：

```
m_cosmosContainer = new QWidget(this);
m_cosmosContainer->setMinimumSize(600, 400);
// 可设置背景色便于调试
m_cosmosContainer->setStyleSheet("background-color:#202020;");

auto *layout  = new QVBoxLayout(central);
layout->addWidget(btnCreate);
layout->addWidget(m_cosmosContainer, 1);
setCentralWidget(central);
```

- **m_cosmosContainer**：后续 Cosmos 窗口就会显示在这个控件区域内。

3.2 获取父窗口句柄并传给 Cosmos

在 `createAndEmbedWidget()` 中，先确保 Qt 容器拥有原生窗口，并取到句柄：

```
// 强制创建原生窗口，并获取其 ID
WId parentWindowId = m_cosmosContainer->winId();
if (!parentWindowId) {
    // 获取失败，直接返回
    return;
}
```

然后按平台转换成字符串，放到 Cosmos 的 `WidgetPreference.ParentHandle` 里：

```
QString parentStr;
#ifdef Q_OS_WIN
    void* hwndPtr = reinterpret_cast<void*>(parentWindowId);
    quintptr hwndValue = reinterpret_cast<quintptr>(hwndPtr);
    parentStr = QString::number(static_cast<qint64>(hwndValue));
#elif defined(Q_OS_LINUX)
    quintptr widValue = reinterpret_cast<quintptr>(parentWindowId);
    parentStr = QString::number(static_cast<unsigned long>(widValue));
#endif

jsPreference.AddMember("ParentHandle",
    rapidjson::Value(parentStr.toStdString().c_str(), doc.GetAllocator()),
    doc.GetAllocator());
```

要点：

- Windows 上 `WId` 就是 HWND 指针类型，因此需要通过 `quintptr` / `void*` 做一次中转后转成整数字符串。
- Cosmos 只关心字符串里的数值，内部再转回平台的句柄类型使用。

3.3 调用 Cosmos 的 CreateWidget

构造 JSON 请求，核心参数包括：

- **WidgetPreference**：
  - `ParentHandle`：父窗口句柄（Qt 容器）
  - `WidgetWidth` / `WidgetHeight`：初始大小
  - `TitleBarVisibility` / `WindowVisibility` / `ResizeMode` 等显示/交互偏好
- **WidgetGuid / AppGuid**：要打开的组件和应用标识。

简化示意：

```
jsParameters.AddMember("WidgetPreference", jsPreference, allocator);
jsParameters.AddMember("WidgetGuid", "组件 GUID", allocator);
jsParameters.AddMember("AppGuid", "应用 GUID", allocator);

jsActionContext.AddMember("Parameters", jsParameters, allocator);
jsActionContext.AddMember("Function", "CreateWidget", allocator);
...

req->Method = "CreateWidget";
req->Parameters = payloadUtf8.c_str();

Cosmos_InvokeResponse* resp = g_invoke(nullptr, req);
```

效果：
Cosmos 引擎收到请求后，会在自己的 GUI 系统里创建一个窗口，其父窗口设置为你传入的 `ParentHandle`，并准备好组件内容。

3.4 解析返回结果，获取 WindowHandle

返回 JSON 中 `ActionContext.Return` 部分包含：

- `WidgetHandle`：逻辑组件句柄。
- `WindowHandle`：实际窗口句柄（字符串形式）。

示意：

```
const auto& returnObj = docResult["ActionContext"]["Return"];

if (returnObj.HasMember("WidgetHandle"))
    m_widgetHandle = returnObj["WidgetHandle"].GetString();

if (returnObj.HasMember("WindowHandle"))
    m_windowHandle = returnObj["WindowHandle"].GetString();
```

这些成员后续会用于：

- **WindowHandle**：Qt 侧嵌入窗口。
- **WidgetHandle + WindowHandle**：调用 `DestroyWidget` 时告知 Cosmos 销毁哪个组件实例。

3.5 将 WindowHandle 转回 WId 并用 Qt 接管

先从字符串转回平台相关 ID：

```
bool ok = false;
WId childWindowId = 0;

#ifdef Q_OS_WIN
    quintptr hwndValue = QString::fromStdString(m_windowHandle).toULongLong(&ok,
10);
    if (ok && hwndValue != 0) {
        void* hwndPtr = reinterpret_cast<void*>(hwndValue);
        childWindowId = reinterpret_cast<WId>(hwndPtr);
    }
#elif defined(Q_OS_LINUX)
    unsigned long x11WindowId =
QString::fromStdString(m_windowHandle).toULong(&ok, 10);
    if (ok && x11WindowId != 0) {
        quintptr widValue = static_cast<quintptr>(x11WindowId);
        childWindowId = reinterpret_cast<WId>(widValue);
    }
#endif
```

然后用 `QWindow::fromWinId` 创建一个 Qt 层面的包装对象：

```
m_embeddedWindow = QWindow::fromWinId(childWindowId);
```

含义：

- `QWindow::fromWinId` 不会创建新窗口，只是用 Qt 的 `QWindow` 来管理一个已有的外部窗口。

3.6 使用 createWindowContainer 放入 Qt 布局

最后一步，把 `QWindow` 包成 `QWidget`，加入之前准备好的容器布局中：

```
m_embeddedWidget = QWidget::createWindowContainer(m_embeddedWindow,
m_cosmosContainer);

if (m_embeddedWidget) {
    QLayout* existingLayout = m_cosmosContainer->layout();
    if (!existingLayout) {
        auto* containerLayout = new QVBoxLayout(m_cosmosContainer);
        containerLayout->setContentsMargins(0, 0, 0, 0);
        containerLayout->setSpacing(0);
        m_cosmosContainer->setLayout(containerLayout);
        containerLayout->addWidget(m_embeddedWidget);
    } else {
        existingLayout->addWidget(m_embeddedWidget);
    }

    m_embeddedWidget->show();
    m_embeddedWidget->setFocus();
}
```

核心要点：

- `createWindowContainer(QWindow*)`：把一个 `QWindow` 包装成可以放进 QWidget 布局体系里的控件。
- 通过把 `m_embeddedWidget` 加入 `m_cosmosContainer` 的布局，实现嵌入显示。
- Qt 会负责同步大小、重绘和部分输入事件，使嵌入窗口看起来像本地控件。

---

4. 生命周期与销毁流程

4.1 销毁组件窗口（DestroyWidget）

当需要关闭组件时（例如重新创建之前）：

1. **调用 Cosmos 的** `DestroyWidget`，传入 `WidgetHandle` + `WindowHandle`，让引擎销毁自己的窗口和相关资源。
2. **Qt 侧清理 UI 对象**：删除 `m_embeddedWidget` / `m_embeddedWindow`，同时清空本地句柄。

示意逻辑：

```
// 1. 通过 SDK 调用 DestroyWidget（略，见代码中的 destroyWidget）

// 2. 本地状态清理
m_widgetHandle.clear();
m_windowHandle.clear();

// 3. UI 清理
if (m_embeddedWidget) {
    m_embeddedWidget->setParent(nullptr);
    delete m_embeddedWidget;   // 会自动删除关联的 m_embeddedWindow
    m_embeddedWidget = nullptr;
    m_embeddedWindow = nullptr;
} else if (m_embeddedWindow) {
    delete m_embeddedWindow;
    m_embeddedWindow = nullptr;
}
```

4.2 主窗口关闭时的顺序

在 `MainWindow::closeEvent` 中，顺序是：

1. **若已创建组件，先调用** `destroyWidget()`（相当于先把嵌入的窗口和 Cosmos 组件收干净）。
2. **再调用** `shutdownCosmos()`，关闭 Cosmos 引擎自身。
3. **最后再允许 Qt 窗口关闭。**

这样可以避免引擎还在跑、窗口句柄还在被引用时，Qt 先把宿主窗口销毁导致各种悬空句柄问题。

---

5. 常见问题与注意事项

- **Q：为什么要先拿父窗口句柄给 Cosmos?**
  A：这样 Cosmos 在创建组件窗口时就能直接把窗口设置为 Qt 容器的子窗口，保证 Z 序、输入焦点和窗口移动时的从属关系都是正确的。

- **Q：如果 `QWindow::fromWinId` 失败怎么办？**
  A：一般是 WindowHandle 解析错误或窗口已被 Cosmos 销毁。需要检查：
  - 字符串转整数的过程是否正确（十进制 / 指针大小）。
  - Cosmos 返回的 WindowHandle 是否为 0 或空字符串。
  - 是否在 Destroy 之后重复使用了同一个句柄。
- **Q：`createWindowContainer` 的生命周期怎么管理？**
  A：Qt 约定：删除 `createWindowContainer` 返回的 `QWidget` 时，会自动删除其内部持有的 `QWindow`。因此：
  - 正常情况只需要 `delete m_embeddedWidget;`。
  - 如果中间某步失败导致只有 `m_embeddedWindow` 而没有 `m_embeddedWidget`，就需要单独 `delete m_embeddedWindow;`。
- **Q：嵌入的 Cosmos 窗口能否自动随 Qt 窗口缩放？**
  A：可以，只要 `m_embeddedWidget` 被放在 Qt 布局中（这里用的是 `QVBoxLayout`），布局会根据宿主窗口大小调整其尺寸，Qt 会同步这个 `QWidget` 对应的 `QWindow` 大小，从而间接调整嵌入窗口。